COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010- Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/3/26	**DATE SUBMITTED:**	10/3/26
NAME:	Rayven Ferry C. Licas	**INSTRUCTOR:**	ENGR. Jimlord M. Quejado

ACTIVITY NO. 10.1 **[** Graphs**]**

1. PROCEDURE OUTPUT

What procedures did you perform?

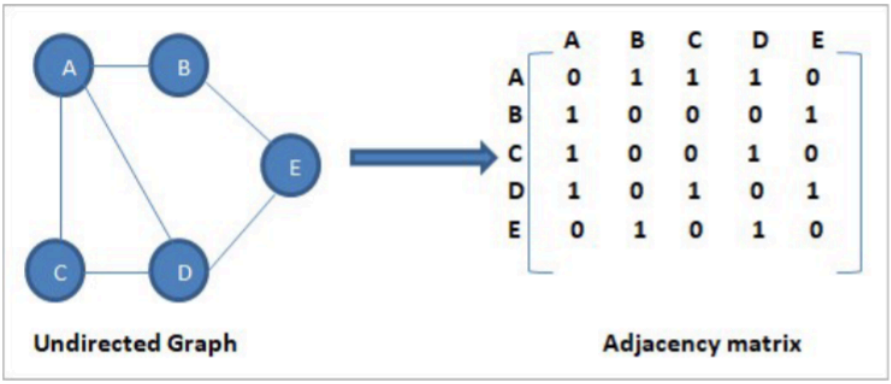
ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

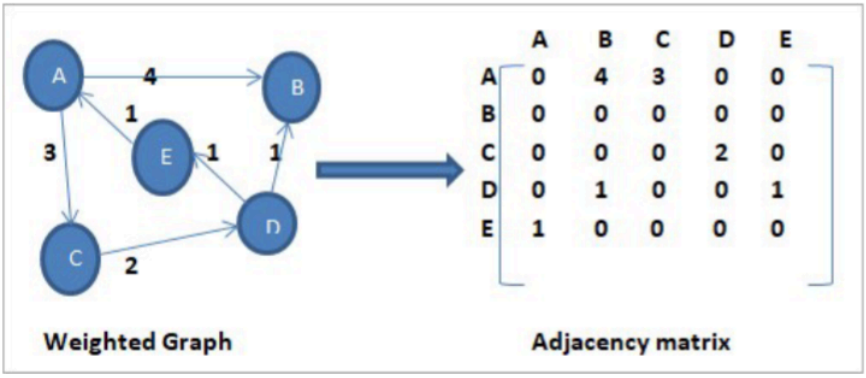
Following are the basic operations that we can perform on the graph data structure:

- **Add a vertex:** Adds vertex to the graph.
- **Add an edge:** Adds an edge between the two vertices of a graph.
- **Display the graph vertices:** Display the vertices of a graph.

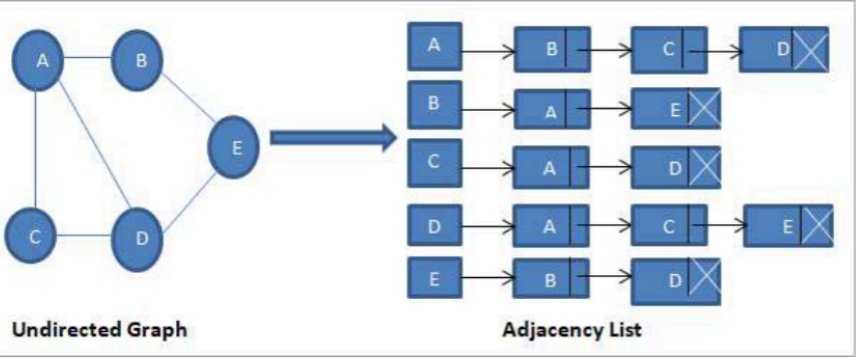
A.2. Adjacency Matrix

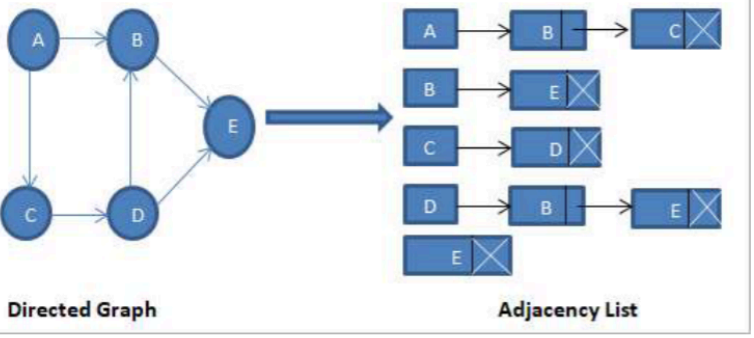


	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	1	0	1	0	1
E	0	1	0	1	0



	A	B	C	D	E
A	0	4	3	0	0
B	0	0	0	0	0
C	0	0	0	2	0
D	0	1	0	0	1
E	1	0	0	0	0


A
B
C
D
E
A B C D
B A E
C A D
D A C E
E B D
Undirected Graph
Adjacency List


A
B
C
D
E
A B C
B E
C D
D B E
E
Directed Graph
Adjacency List

Sample Code:

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head)    {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head;   // point new node to current head
```



```
        return newNode;
    }
    int N;  // number of nodes in the graph
public:
    adjNode **head;               //adjacency list as array of pointers
    // Constructor
```

```
        DiaGraph(graphEdge edges[], int n, int N)  {
            // allocate new node
            head = new adjNode*[N]();
            this->N = N;
            // initialize head pointer for all vertices
            for (int i = 0; i < N; ++i)
                head[i] = nullptr;
            // construct directed graph by adding edges to it
            for (unsigned i = 0; i < n; i++)  {
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;
                // insert in the beginning
                adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

                // point head pointer to new node
                head[start_ver] = newNode;
            }
        }
        // Destructor
        ~DiaGraph() {
            for (int i = 0; i < N; i++)
                delete[] head[i];
            delete[] head;
        }
    };
    // print all adjacent vertices of given vertex
    void display_AdjList(adjNode* ptr, int i)
    {
        while (ptr != nullptr) {
            std::cout << "(" << i << ", " << ptr->val
                 << ", " << ptr->cost << ") ";
            ptr = ptr->next;
        }
        std::cout << std::endl;
    }
    // graph implementation
    int main()
    {
        // graph edges array.
        graphEdge edges[] = {
                // (x, y, w) -> edge from x to y with weight w
                {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
        };
        int N = 6;      // Number of vertices in the graph
        // calculate number of edges
        int n = sizeof(edges)/sizeof(edges[0]);
        // construct graph
```

6. Output

7. Supplementary Activity

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

3. In the performed code, what data structure is used to implement the Breadth First Search?

4. How many times can a node be visited in the BFS?

8. Conclusion

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Vertex IndexConsole OutputDescription0(0, 2, 4) (0, 1, 2)
Neighbors connected from vertex 0

1(1, 4, 3)
Neighbor connected from vertex 1

2(2, 3, 2)
Neighbor connected from vertex 2

3(3, 1, 4)
Neighbor connected from vertex 3

4(4, 3, 3)
Neighbor connected from vertex 4

5(empty line)

No outgoing edges from vertex 5

ComponentOutput / Result / ObservationWeighted Graph Structure

1: {2: 2}, {5: 3},

2: {1: 2}, {5: 5}, {4: 1},

3: {4: 2}, {7: 3},

4: {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},

5: {1: 3}, {2: 5}, {4: 2}, {8: 3},

6: {4: 4}, {7: 4}, {8: 1},

7: {3: 3}, {6: 4},

8: {4: 5}, {5: 3}, {6: 1},

BFS Traversal Sequence

1 -> 2 -> 5 -> 4 -> 8 -> 3 -> 6 -> 7

Traversal Behavior Observation

BFS explores nodes layer-by-layer (level-wise). From source 1, it first visits level-1 neighbors (2, 5), then level-2 ne\ighbors (4, 8), followed by level-3 neighbors (3, 6), and finally level-4 (7).

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The most helpful algorithm is Depth-First Search (DFS). It goes deep into one path first, then backtracks when there are no more vertices to visit. This matches the situation described in the question.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The equivalent of DFS in tree traversal is Depth-First Tree Traversal. Its common methods are Preorder, Inorder, and Postorder traversal. DFS uses a stack or recursion to go deeper into the tree before backtracking.

3. In the performed code, what data structure is used to implement the Breadth First Search?

The data structure used for Breadth-First Search is a Queue. The vertex is added using enqueue() and removed using dequeue().

4. How many times can a node be visited in the BFS?
A node should be visited only once in BFS. The program uses a `visited` set to keep track of nodes that have already been visited.

Conclusion

The activity helped me understand how graphs are represented using adjacency lists and how graph traversal works. I learned that DFS uses a stack and goes deeper before backtracking, while BFS uses a queue and visits nodes level by level. I also learned how visited nodes prevent repeated processing. The supplementary activity helped me understand the difference between DFS and BFS and the data structures they use.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.